Department of Computer Science & Engineering
The University of Texas at Arlington

# Architectural Design Specification
# CSE 4316: Senior Design I
# Fall 2025

ByteBuddi
Build Better Resumes

## Team ByteBuddi
## ByteBuddi

Adalynn Everly
Ryan Davis
Alan Acevedo
Xavior Lopez
Rene Salazar
Cesar Gallegos

## Revision History

| Revision | Date | Author(s) | Description |
|---|---|---|---|
| 0.1 | 11.7.2025 | RS, AE, CG, XL, AA, RD | Initial Submission |
| | | | |
| | | | |
| | | | |
| | | | |

# Contents


**1 Introduction** **5**

**2 System Overview** **6**
2.1 Presentation Layer Description . . . . . 7
2.2 Application Layer Description . . . . . 7
2.3 Data and Integration Layer Description . . . . . 7

**3 Subsystem Definitions & Data Flow** **8**

**4 Presentation Layer Subsystems** **9**
4.1 Subsystem 1-Authentication UI . . . . . 9
4.2 Subsystem 2-Resume Builder UI . . . . . 10
4.3 Assumptions . . . . . 10
4.4 Responsabilities . . . . . 10
4.5 Subsystem 3-Templates and Preview UI . . . . . 12
4.6 Assumptions . . . . . 12
4.7 Responsibilities . . . . . 12
4.8 Subsystem 4 - Interview UI . . . . . 13
4.9 Responsibilities . . . . . 13

**5 APPLICATION LAYER SUBSYSTEMS** **14**
5.1 Subsystem 1-Authentication and Session Management . . . . . 14
5.2 Subsystem 2-Resume Data Management . . . . . 15
5.3 Assumptions . . . . . 15
5.4 Responsabilities . . . . . 15
5.5 Subsystem 3-AI Interaction and PDF Export . . . . . 17
5.6 Assumptions . . . . . 17
5.7 Responsabilities . . . . . 17

**6 Data and Integration Layer Subsystems** **18**
6.1 Subsystem 1 - Identity and Access . . . . . 18
6.2 Subsystem 2 - Firestore Data Store . . . . . 19
6.3 Subsystem 3 - External Providers and Storage . . . . . 20

## List of Figures


| | | |
|---|---|---|
| 1 | ByteBuddi Three-Layer Architecture Diagram | 6 |
| 2 | ByteBuddi Data Flow Diagram | 8 |
| 3 | Presentation Layer Data Flow Diagram | 9 |
| 4 | Application Layer Data Flow Diagram | 14 |
| 5 | Data and Integration Layer Flow Diagram | 18 |


## List of Tables


| | | |
|---|---|---|
| 2 | Subsystem interfaces | 10 |
| 3 | Subsystem interfaces | 11 |
| 4 | Subsystem interfaces | 12 |
| 5 | Subsystem interfaces | 13 |
| 6 | Subsystem interfaces | 15 |
| 7 | Subsystem interfaces | 16 |
| 8 | Subsystem interfaces | 17 |
| 9 | Subsystem interfaces | 19 |
| 10 | Subsystem interfaces | 20 |
| 11 | Subsystem interfaces | 20 |

# 1 INTRODUCTION

The Automated Resume Builder is a web application that enables college students and entry-level job seekers to create, edit, preview, and export resumes for internships and entry-level roles. The product supports account management, multiple resume templates, AI-driven mock interviews, admin analytics, and integration with third-party career platforms (e.g Handshake).

This architectural document describes the system structure, the three logical layers, their subsystems, and the data flow between them. The design emphasizes modularity, security, scalability, and maintainability aligned with the project sprint plan and non functional requirements (performance, availability, and safety). Key requirements driving this architecture:

- Secure user authentication and data storage (Safety-1, Safety-2).
- Responsive resume preview and PDF export within performance budgets (Perf-1, Perf-2).
- AI-based interview flows and integrations (Mock Interviews, Handshake).
- Admin analytics and template management.

## 2 System Overview

The system is designed as a modular web-based platform that helps college students and entry-level professionals efficiently create a professional resume and practice for interviews through AI-assisted features.

The system follows a three-tier layered architecture:

- **Presentation Layer (Client):** React single-page app providing the UI for users/admins; routes include Home, Gallery, Builder, Templates, About, Login, and Interviewer.
- **Application Layer (Backend):** Node.js/Express services implementing business logic: authentication, resume management, template engine, PDF generation, AI Management, admin APIs.
- **Data and Integration Layer:** Persistent stores, object storage for assets, cache/message layer, and external APIs (AI provider, Handshake).

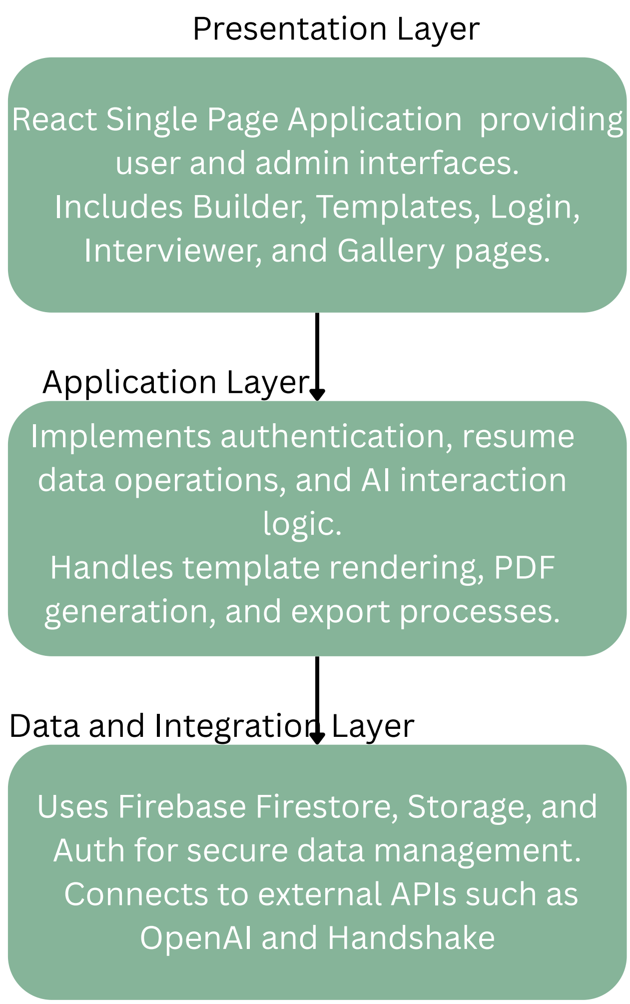


Figure 1: ByteBuddi Three-Layer Architecture Diagram

## 2.1 PRESENTATION LAYER DESCRIPTION

The Presentation Layer of ByteBuddi is implemented as a React single-page application that serves as the primary interface for the users and administrators. It manages every client interaction, state transitions, and communication with backend APIs and Firebase services. The navigation between pages such as Home, Builder, Templates, Login, and Interviewer is handled through React Router for a dynamic and seamless experience. This first layer is responsible for rendering resume templates in real time, validating user input, starting AI interview sessions, and providing feedback through a responsive UI.

## 2.2 APPLICATION LAYER DESCRIPTION

The application layer is implemented using Firebase services and logic rather than a dedicated backend server. It handles authentication using Firebase Auth, manages data transactions with Firestore, and coordinates resume storage and retrieval from our database. This layer is also responsible for connecting external APIs such as OpenAI for mock interviews, and integrates export utilities that generate PDF files from user templates.

## 2.3 DATA AND INTEGRATION LAYER DESCRIPTION

The Data and Integration Layer provides storage, identity management, and external connectivity. It uses Firebase Firestore for structured resume and user data, Firebase storage for data and exported PDFs, and Firebase Auth for secure user identification. This layer also integrates OpenAI for Ai interview smulations. It ensures data remains consistent, synchronized, and it secures access control, serving as the foundation for ByteBuddi's cloud-based data architecture.

## 3 Subsystem Definitions & Data Flow

The ByteBuddi system is made up of interdependent subsystems distributed across the presentation, application, and data layers. The presentation layer handles user interaction, collection and validation of resume input, template rendering, and initiating AI interviews. The application layer, implemented through Firebase, manages authentication, synchronization with data, and communication with the external API. The data and integration layer provides connectivity through Firebase Firestore, storage, and Auth, ensuring that all information and user credentials are safely stored and retrievable. The data flows in both directions between the subsystems: user actions trigger updates to Firestore, AI requests generate the responses in the UI, and the export operations retrieve and store information in Firebase Storage, maintaining a successful coordination throughout the system.

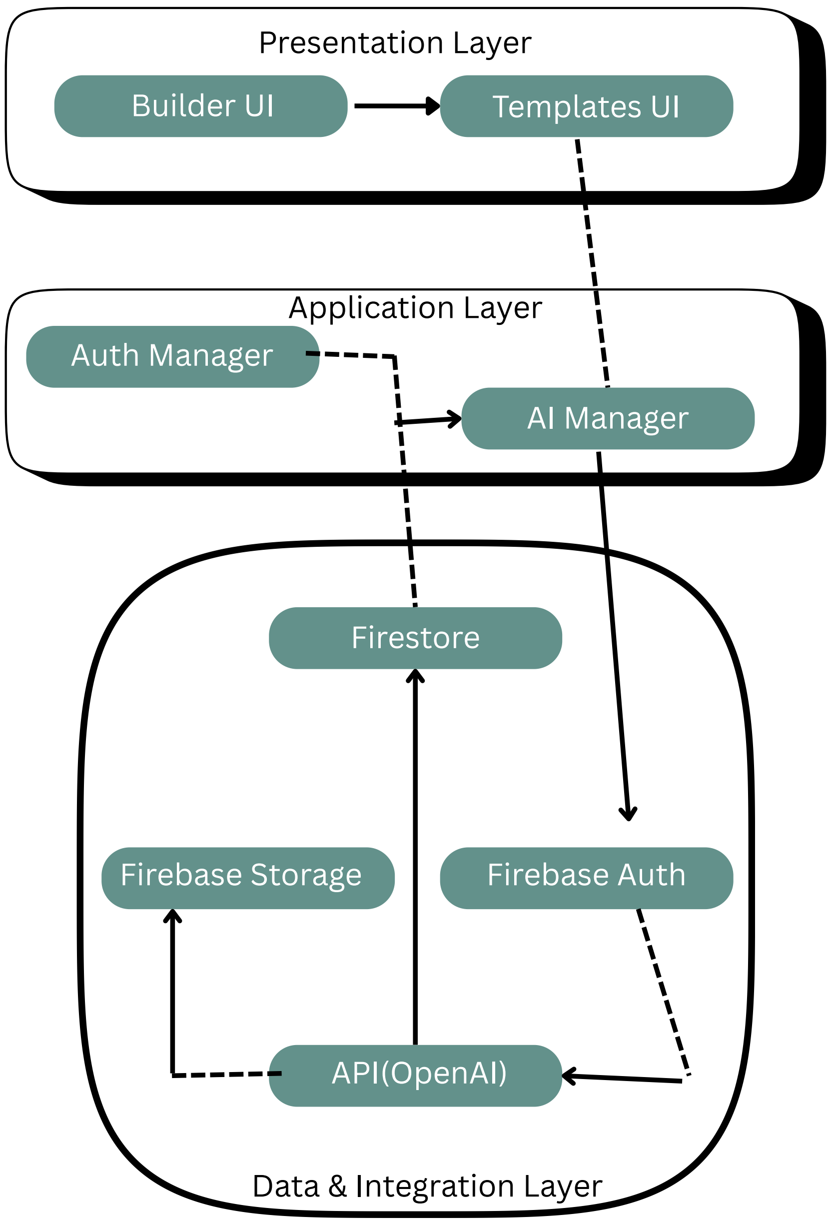


Figure 2: ByteBuddi Data Flow Diagram

# 4 Presentation Layer Subsystems

## 4.1 Subsystem 1-Authentication UI

This section describes the main subsystems of the Presentation Layer. Each subsystem represents a key component of the single page application, responsible for collecting user data, managing templates, and initiating AI interactions.

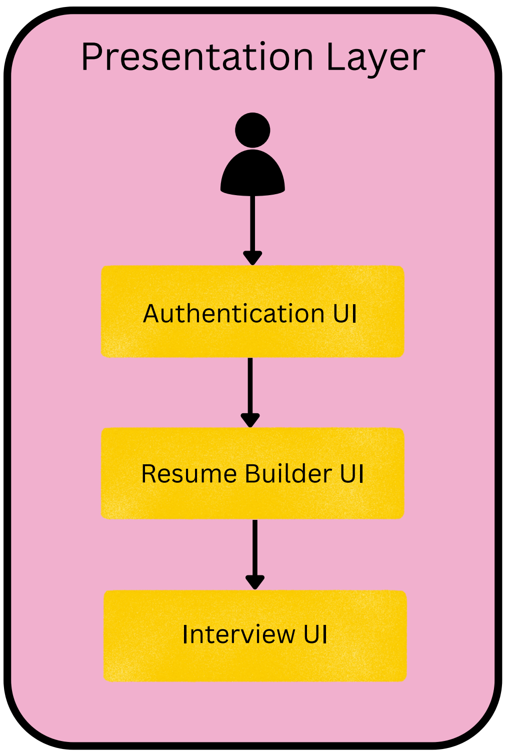


Figure 3: Presentation Layer Data Flow Diagram

### 4.1.1 Assumptions

- Firebase Authentication service is configured and available.
- HTTPS communication is enforced for all user credentials.
- Session remains stable and the tokens are refreshed automatically.

### 4.1.2 Responsibilities

- Display and validate login, registration, and password reset.
- Send authentication requests to Firebase Auth.
- Display error or success messages for authentication events.

### 4.1.3 Subsystem Interfaces

Table 2: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| #x1 | Firebase Auth SDK | Email, password, or OAuth token | Auth token, user ID |
| #x2 | Route Guard | Auth state | Redirect or allow access |

## 4.2 Subsystem 2-Resume Builder UI

This Resume Builder subsystem provides an interactive interfacee for creating, editing, and previewing resumes. It includes inputs for personal details, education, experience, and skills. The builder automatically validates the input and synchronizes changes with the database in real time.

## 4.3 Assumptions

- Firestore security rules only allow the authenticated users to edit their resumes.
- The connection between the UI and Firsetore is stable and secure.
- Data follows a predefined schema.

## 4.4 Responsabilities

- Display a form based resume creation sections.
- Validate and autosave data to Firestore.
- Support editing and deletion of resume entries.
- Provide feedback for missing or invalid inputs.

Table 3: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| #x3 | Firestore Resume Document | Form data | Updated resume document |
| #x4 | Schema Validator | Resume | Validation errors or normalized data |

## 4.5 Subsystem 3-Templates and Preview UI

This subsystem allows users to select and preview resume templates in real time. It retrieves available templates from the database and renders based on the choice. The users can switch between templates and view updates before exporting.

## 4.6 Assumptions

- Templates are stored and preloaded in Firestore/Storage.
- Rendering remains consistent across browsers and devices.
- Template preview updates dynamically when the data changes.

## 4.7 Responsibilities

- Display template gallery preview.
- Render live resume preview using the selected template.
- Communicate with export utilities to generate a downloadable file.

Table 4: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| #x5 | Template Loader | Template ID | Template structure and metadata |
| #x6 | Preview Renderer | Resume data and template | Live preview view |

## 4.8 Subsystem 4 - Interview UI

This subsystem supports mock AI interviews by first collecting context data from the users (job, role, skills, etc) and displaying dynamically generated questions. It communicates with the AI manager in the application layer and provides real time responses and feedback.

### 4.8.1 Assumptions

- The AI manager service is online and has been authenticated.
- Responses are returned effectively in text format.

## 4.9 Responsibilities

Receive interview context and send it to the AI manager. Display questions, capture responses effectively, and provide useful feedback. Log interview sessions to Firestore for review.

Table 5: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| #x7 | AI Manager API | Context data, resume highlights | Generated questions, feedback |
| #x8 | Transcript Logger | Session data | Stored transcript document |

## 5 APPLICATION LAYER SUBSYSTEMS

In this section, the ByteBuddi application layer is described in more detail through the main subsystems. This layer manages application logic, authentication, data coordination, and AI services. Each subsystem is responsible for a specific set of operations, ensuring modularity, maintainability, and scalability of the system. The following sections will describe these subsystems, their functions, and interfaces.

### 5.1 Subsystem 1-Authentication and Session Management

This subsystem manages user authentication and authorization. It uses Firebase Authentication to register and log users in safely, providing validation and token management, it also handles user roles (Admin, Standard User) and integrates with Firestore to keep everything consistent.

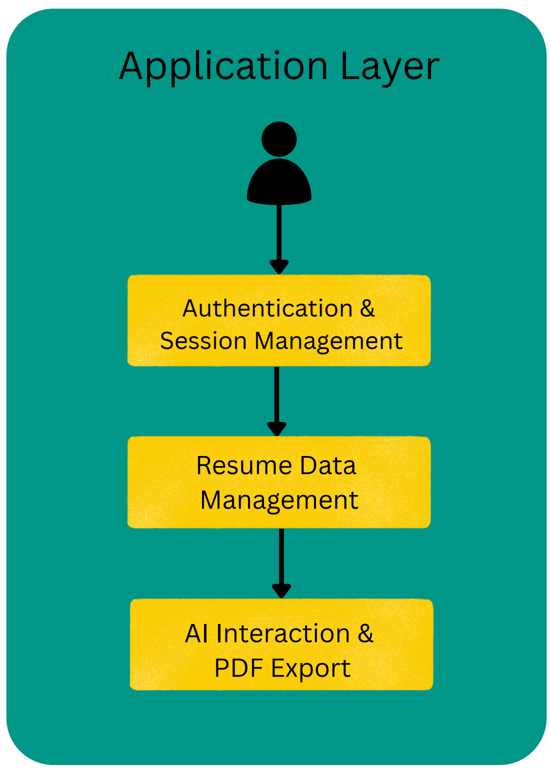


Figure 4: Application Layer Data Flow Diagram

### 5.1.1 Assumptions

- Firebase Auth service is working properly and configured correctly.
- Tokens will be refreshed automatically before they expire.
- All user credentials will be transmitted over HTTPS.

### 5.1.2 Responsibilities

- Authenticate the users and manage session tokens.
- Handle password reset and user registration.
- Manage role based access.
- Communicate safely with Firestore and other Firebase subsystems.

### 5.1.3 Subsystem Interfaces

Table 6: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| #A1 | Firebase Authentication API | email, password | token |
| #A2 | Role verification service | user ID | access level |
| #A3 | Session handler | token | session state |

## 5.2 Subsystem 2-Resume Data Management

This subsystem is responsible for handling all operations related to creating, updating, and storing user resume data. It interacts with Firestore for document storage and supports template mapping, version control, and export.

## 5.3 Assumptions

- Firestore permissions are correcly defined per user.
- Templates and resume data remains consistent.
- Resume operations can trigger updates (autosaves).

## 5.4 Responsabilities

- Update,create, and retrieve the user resume entries.
- Map the templates.
- Track user changes and autosave progress.
- Send data to the export subsystem for PDF generation.

Table 7: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| #R1 | Firestore operations | resume ID, data payload | success/failure status |
| #R2 | Template mapper | user data | formatted resume object |
| #R3 | Export trigger | formatted resume object | render request |

## 5.5 SUBSYSTEM 3-AI INTERACTION AND PDF EXPORT

This subsystem connects the system to the external AI and export utilities, it sends resume information to OpenAI for interview questions generation and communicates with the PDF renderer for downloadable resumes.

## 5.6 ASSUMPTIONS

- API keys and Firebase storage are valid..
- AI responses are filtered and formatted before display.
- PDF generation happens asynchronously to avoid UI delays.

## 5.7 RESPONSABILITIES

- Send context to OpenAI API for moch interviews.
- Receive and format AI-generated questions and fedback.
- Generate and export PDF versions of resumes to the Firebase storage.
- Notify users when the export or AI feedback is complete.

Table 8: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| #I1 | OpenAI API interface | resume data | generated Q&A |
| #I2 | PDF generation utility | formatted resume | PDF file |
| #I3 | Notification handler | process state | user alert |

# 6 Data and Integration Layer Subsystems

In this section, the Z Layer is described in detail. This layer owns identity, storage, and external providers. It includes Firebase Auth, Firestore, Storage, and AI or third party API connections. The Application Layer does not talk to these services directly. All traffic is routed through the subsystem.

## 6.1 Subsystem 1 - Identity and Access

This subsystem handles login and security. Firebase Auth stores credentials, issues tokens, and enforces access rules. The Application Layer checks every request to make sure the user is logged in and allowed to reach the resource.

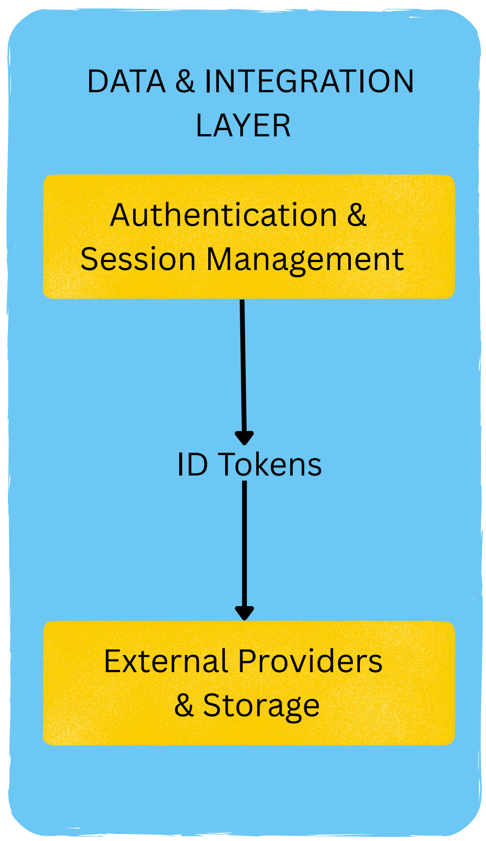


Figure 5: Data and Integration Layer Flow Diagram

### 6.1.1 Assumptions

- HTTPS is always used.
- The tokens expire and are refreshed automatically on the client.
- Role information is stored in Auth claims or inside Firestore.
- Only verified `mavs.uta.edu` accounts are allowed.

### 6.1.2 Responsibilities

- Create accounts, sign in users, and reset passwords.
- Issue and verify ID tokens for the Application Layer.
- Enforce allowed roles such as Admin.
- Provide UUIDs for user-specific data storage.

### 6.1.3 Subsystem Interfaces

Table 9: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| Z1 | Firebase Auth login and registration | email, password | token, user ID |
| Z2 | Token verification for backend requests | token | allow or deny |
| Z3 | Auth claims lookup | user ID | role claims |

## 6.2 Subsystem 2 - Firestore Data Store

This subsystem stores user resumes, template data, and system settings. The Application Layer never writes files directly. Every document write or read flows through this subsystem. Resume updates and template previews depend on this storage.

### 6.2.1 Assumptions

- Security rules prevent users from reading or editing other users' data.
- Only logged in users may write resume data.
- Data follows the project schema stored in the sprint documentation.
- Real-time listeners return updates to the front-end.

### 6.2.2 Responsibilities

- Create, update, and retrieve user resume documents.
- Store template metadata and shared assets.
- Manage version tags for resume rollback.
- Support real-time updates for live preview.

### 6.2.3 Subsystem Interfaces

Table 10: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| Z4 | Firestore CRUD | user ID, payload | success or error |
| Z5 | Real-time snapshot listener | query | change events |
| Z6 | Security rules engine | token, document path | allow or deny |

## 6.3 Subsystem 3 - External Providers and Storage

This subsystem handles Firebase Storage for file uploads and the OpenAI API for interview questions. It creates PDFs and stores them for download. This keeps API keys and sensitive operations on the server side.

### 6.3.1 Assumptions

- API keys stay on the server only.
- Storage objects such as exported PDFs may be deleted after 30 days.
- Only minimal resume data are sent to AI providers.

### 6.3.2 Responsibilities

- Generate and store PDF's for download.
- Store template images, CSS, and public assets.
- Send resume highlights to OpenAI for mock interview prompts.
- Filter and format AI responses for the Application Layer.

### 6.3.3 Subsystem Interfaces

Table 11: Subsystem interfaces

| ID | Description | Inputs | Outputs |
|---|---|---|---|
| Z7 | Firebase Storage upload | blob, file path | download URL |
| Z8 | PDF renderer service | formatted resume | PDF file |
| Z9 | OpenAI request handler | prompt data | interview questions and feedback |
| Z10 | Optional export to Handshake | resume file and metadata | provider status |

# References